DOKUMENTASI PEMBELAJARAN

Integrasi API RajaOngkir & Proses Checkout Produk

Proyek: Aplikasi TokoOnline (Laravel 10)

Mata Kuliah: Web Programming III

Materi: Minggu ke-7 & Minggu ke-9

Framework: Laravel 10 • PHP 8.1 • MySQL (Laragon)

Referensi Modul: 0688_Modul-Web Programming III

Status: Implementasi selesai & teruji

> Dokumen ini disusun dari kode yang benar-benar berjalan di proyek, bukan hanya salinan modul. Perbedaan penting antara modul dan implementasi dijelaskan pada setiap bagian agar mudah dipelajari.

Daftar Isi

Bagian 1 — Gambaran Umum

1.1 Tujuan Dokumen
1.2 Prasyarat
1.3 Alur Pengguna
1.4 Peta File
1.5 Peta Route

Bagian 2 — Minggu ke-7: Integrasi API RajaOngkir

2.1 Tujuan Pembelajaran
2.2 Konsep RajaOngkir dan Komerce
2.3 Konfigurasi Environment
2.4 Arsitektur Controller
2.5 Mengambil Data Provinsi
2.6 Mengambil Data Kota
2.7 Menghitung Ongkos Kirim
2.8 Normalisasi Respons API
2.9 Caching Data Ongkir
2.10 Penanganan Error
2.11 Route Ongkir
2.12 Halaman Cek Ongkir
2.13 Pengujian Minggu 7
2.14 Perbedaan Modul vs Implementasi
2.15 Latihan

Bagian 3 — Minggu ke-9: Proses Checkout Produk

3.1 Tujuan Pembelajaran
3.2 Alur Checkout dan Status Pesanan
3.3 Database dan Model
3.4 Memilih Pengiriman
3.5 Menyimpan Ongkir dengan Revalidasi Server
3.6 Halaman Pembayaran
3.7 Checkout dan Pengurangan Stok
3.8 History Pesanan
3.9 Route Checkout
3.10 Tampilan Checkout
3.11 Perbaikan Layout Tombol
3.12 Catatan Payment Gateway
3.13 Pengujian Minggu 9
3.14 Perbedaan Modul vs Implementasi
3.15 Latihan

Bagian 4 — Ringkasan dan Operasional

4.1 Ringkasan Alur
4.2 Troubleshooting
4.3 Checklist Belajar
4.4 Command Reference
4.5 Daftar File
4.6 Catatan Keamanan

Lampiran

A. Contoh Request dan Response Komerce
B. Contoh Response RajaOngkir Starter
C. Struktur Tabel Order
D. Ringkasan Hasil Pengujian

Bagian 1 — Gambaran Umum

1.1 Tujuan Dokumen

Dokumen ini dibuat sebagai bahan belajar untuk memahami dua fitur utama pada aplikasi **TokoOnline**:

1. **Integrasi API RajaOngkir** untuk mengambil data provinsi, kota, dan ongkos kirim.
2. **Proses checkout produk** mulai dari memilih pengiriman, menghitung ongkir, menampilkan pembayaran, sampai menyimpan history pesanan.

Setelah membaca dokumen ini, diharapkan pembaca mampu:

- Menjelaskan alur request dari browser ke API RajaOngkir.
- Memahami perbedaan format respons Komerce dan RajaOngkir Starter.
- Menambahkan validasi, caching, dan penanganan error pada integrasi API.
- Memahami alur checkout yang aman: validasi stok, transaksi database, dan revalidasi biaya ongkir di server.
- Menjalankan pengujian otomatis untuk memastikan alur checkout tidak rusak.

> **Inti materi:** jangan hanya menyalin kode. Pahami *mengapa* biaya ongkir harus dihitung ulang di server, *mengapa* stok harus dikunci saat checkout, dan *mengapa* status pesanan harus konsisten dengan halaman history.

1.2 Prasyarat

- PHP 8.1 atau lebih baru.
- Laravel 10 sudah terpasang.
- MySQL/MariaDB melalui Laragon sudah berjalan.
- Database `db_tokoonline` sudah dibuat dan dimigrasikan.
- Ekstensi PHP `curl` dan `openssl` aktif.
- Akun Komerce/RajaOngkir dan API key yang valid.
- Koneksi internet saat menguji API asli.

Perintah dasar untuk memeriksa proyek:

```
php artisan about
php artisan migrate:status
php artisan route:list
```

1.3 Alur Pengguna

Alur utama yang dibangun pada Minggu 7 dan Minggu 9:

```
Login Google
    |
    v
Halaman Produk
    |
    v
Tambah ke Keranjang
    |
```

```
    v
Keranjang Belanja
    |
    v
Pilih Pengiriman
    |
    +--> Pilih Provinsi
    +--> Pilih Kota
    +--> Pilih Kurir
    +--> Cek Ongkir (API RajaOngkir)
    +--> Pilih Layanan (REG, YES, dll.)
    |
    v
Halaman Pembayaran
    |
    +--> Subtotal Produk
    +--> Ongkos Kirim
    +--> Total Bayar
    |
    v
Konfirmasi Bayar
    |
    v
History Pesanan (status Paid / Kirim / Selesai)
```

1.4 Peta File

Kebutuhan	File
Controller ongkir	`app/Http/Controllers/RajaOngkirController.php`
Exception ongkir	`app/Exceptions/RajaOngkirException.php`
Controller checkout	`app/Http/Controllers/OrderController.php`
Model pesanan	`app/Models/Order.php`
Konfigurasi layanan	`config/services.php`
Route	`routes/web.php`
Halaman cek ongkir	`resources/views/ongkir.blade.php`
Halaman pilih pengiriman	`resources/views/v_order/select_shipping.blade.php`
Halaman pembayaran	`resources/views/v_order/selectpayment.blade.php`
Halaman history	`resources/views/v_order/history.blade.php`
Test ongkir	`tests/Feature/RajaOngkirTest.php`
Test checkout	`tests/Feature/OrderCheckoutTest.php`

1.5 Peta Route

Method	URI	Name	Fungsi
GET	`/cek-ongkir`	`ongkir.index`	Halaman uji cek ongkir
GET	`/provinces`	`ongkir.provinces`	Ambil daftar provinsi
GET	`/cities`	`ongkir.cities`	Ambil daftar kota
POST	`/cost`	`ongkir.cost`	Hitung ongkos kirim
GET/POST	`/select-shipping`	`order.selectShipping`	Halaman pilih pengiriman
POST	`/updateongkir`	`order.updateongkir`	Simpan ongkir pilihan
GET	`/selectpayment`	`order.selectPayment`	Halaman pembayaran
POST	`/checkout`	`order.checkout`	Konfirmasi bayar dan kurangi stok
GET	`/history`	`order.history`	Riwayat pesanan customer

Bagian 2 — Minggu ke-7: Integrasi API RajaOngkir

2.1 Tujuan Pembelajaran

Setelah mempelajari bagian ini, kita mampu:

- Mengambil data provinsi dan kota dari API RajaOngkir.
- Menghitung ongkos kirim berdasarkan kota asal, kota tujuan, berat, dan kurir.
- Menangani dua format respons API: Komerce dan RajaOngkir Starter.
- Menambahkan caching agar API tidak dipanggil berulang kali.
- Menampilkan pesan error yang jelas saat API key salah atau koneksi gagal.

2.2 Konsep RajaOngkir dan Komerce

RajaOngkir adalah layanan yang menyediakan data ongkos kirim dari berbagai ekspedisi seperti JNE, TIKI, dan POS Indonesia. Data yang dibutuhkan aplikasi e-commerce biasanya:

- Daftar provinsi.
- Daftar kota/kabupaten per provinsi.
- Biaya pengiriman per layanan.
- Estimasi waktu pengiriman (ETD).

Saat ini API RajaOngkir dapat diakses melalui **Komerce**. Perbedaan penting yang harus dipahami:

Aspek	Komerce	RajaOngkir Starter
Base URL	`https://rajaongkir.komerce.id/api/v1`	`https://api.rajaongkir.com/starter`
Endpoint provinsi	`/destination/province`	`/province`
Endpoint kota	`/destination/city/{provinceId}`	`/city?province={id}`
Endpoint biaya	`/calculate/domestic-cost`	`/cost`
Bentuk data	`{ meta, data }`	`{ rajaongkir: { status, results } }`
Pengiriman biaya	form request	form request

Karena bentuk responsnya berbeda, controller dibuat agar **mendeteksi otomatis** jenis API berdasarkan base URL. Dengan begitu, kode yang sama bisa dipakai untuk Komerce maupun Starter.

> **Catatan:** jangan menulis API key di dalam kode, dokumen, atau repositori. Simpan key di `.env` dan pastikan `.env` tidak ikut di-commit.

2.3 Konfigurasi Environment

Tambahkan konfigurasi berikut pada `.env`:

```
RAJAONGKIR_API_KEY=your_api_key
RAJAONGKIR_BASE_URL=https://rajaongkir.komerce.id/api/v1
RAJAONGKIR_TIMEOUT=15
RAJAONGKIR_ORIGIN_CITY_ID=137
RAJAONGKIR_ORIGIN_CITY_NAME="Jakarta Pusat"
```

Penjelasan:

- `RAJAONGKIR_API_KEY` — key dari Komerce/RajaOngkir.
- `RAJAONGKIR_BASE_URL` — alamat API yang dipakai.
- `RAJAONGKIR_TIMEOUT` — batas waktu request dalam detik.
- `RAJAONGKIR_ORIGIN_CITY_ID` — ID kota asal toko.
- `RAJAONGKIR_ORIGIN_CITY_NAME` — nama kota asal untuk ditampilkan.

> **Penting:** nilai yang mengandung spasi seperti `Jakarta Pusat` harus diberi tanda kutip. Jika tidak, Laravel dapat gagal membaca `.env` dengan pesan `Encountered unexpected whitespace`.

Nilai tersebut dibaca melalui `config/services.php`:

```
'rajaongkir' => [
    'key' => env('RAJAONGKIR_API_KEY'),
    'base_url' => env('RAJAONGKIR_BASE_URL', 'https://api.rajaongkir.com/starter'),
    'timeout' => (int) env('RAJAONGKIR_TIMEOUT', 15),
    'origin_city_id' => env('RAJAONGKIR_ORIGIN_CITY_ID', '137'),
    'origin_city_name' => env('RAJAONGKIR_ORIGIN_CITY_NAME', 'Jakarta Pusat'),
],
```

Mengapa memakai `config()` dan bukan `env()` langsung di controller?

- `env()` hanya boleh dipakai di file konfigurasi.
- Setelah `php artisan config:cache`, nilai `env()` di luar config bisa bernilai `null`.
- `config()` membuat kode lebih mudah diuji karena nilainya bisa diubah saat test.

2.4 Arsitektur Controller

`RajaOngkirController` memiliki tiga method publik:

Method	Fungsi
`getProvinces()`	Mengambil daftar provinsi
`getCities()`	Mengambil daftar kota berdasarkan provinsi
`getCost()`	Menghitung ongkos kirim

Method privat membantu memisahkan tanggung jawab:

Method privat	Fungsi
`fetchProvinces()`	Request ke API dan normalisasi provinsi
`fetchCities()`	Request ke API dan normalisasi kota
`fetchCosts()`	Request ke API dan normalisasi biaya
`request()`	Menangani response HTTP dan error API
`handle()`	Membungkus eksekusi dan mengubah exception menjadi JSON
`success()`	Format respons sukses yang konsisten
`error()`	Format respons error yang konsisten
`client()`	Membuat HTTP client dengan key dan timeout
`isKomerce()`	Mendeteksi jenis API dari base URL
`cacheKey()`	Membuat cache key yang aman

Semua method publik mengembalikan `JsonResponse` dengan bentuk yang sama:

```
{
  "status": { "code": 200, "description": "OK" },
  "data": []
}
```

Dengan bentuk yang konsisten, JavaScript di frontend tidak perlu menebak format respons API.

2.5 Mengambil Data Provinsi

```
public function getProvinces(): JsonResponse
{
    return $this->handle(function (): JsonResponse {
        $provinces = Cache::remember(
            $this->cacheKey('provinces'),
            now()->addDay(),
            fn (): array => $this->fetchProvinces()
        );

        return $this->success($provinces);
    });
}
```

Alurnya:

1. `handle()` memeriksa apakah API key sudah diisi.
2. `Cache::remember()` mencari data di cache.
3. Jika belum ada, `fetchProvinces()` memanggil API.
4. Data disimpan selama satu hari.
5. `success()` mengembalikan JSON dengan format standar.

Bagian `fetchProvinces()` untuk Komerce:

```
$payload = $this->request(
    fn (): Response => $this->client()->get($this->baseUrl().'/destination/province')
);
```

```
return collect($this->dataItems($payload))
    ->map(fn (array $item): array => [
        'province_id' => (string) ($item['id'] ?? ''),
        'province' => (string) ($item['name'] ?? ''),
    ])
    ->filter(fn (array $item): bool => $item['province_id'] !== '' && $item['province'] !== '')
    ->values()
    ->all();
```

Untuk RajaOngkir Starter, data diambil dari `rajaongkir.results` lalu dipetakan ke bentuk yang sama.

2.6 Mengambil Data Kota

```
public function getCities(Request $request): JsonResponse
{
    $validated = $request->validate([
        'province_id' => ['required', 'integer', 'min:1'],
    ], [
        'province_id.required' => 'ID provinsi wajib diisi.',
        'province_id.integer' => 'ID provinsi harus berupa angka.',
        'province_id.min' => 'ID provinsi tidak valid.',
    ]);

    $provinceId = (string) $validated['province_id'];

    return $this->handle(function () use ($provinceId): JsonResponse {
        $cities = Cache::remember(
            $this->cacheKey('cities.'.$provinceId),
            now()->addDay(),
            fn (): array => $this->fetchCities($provinceId)
        );

        return $this->success($cities);
    });
}
```

Perhatikan:

- `province_id` divalidasi sebelum dikirim ke API.
- Cache key menyertakan `provinceId`, sehingga kota provinsi A tidak tertukar dengan provinsi B.
- Data kota Komerce dinormalisasi menjadi:

```
{
  "city_id": "1",
  "province_id": "137",
  "province": "",
  "type": "",
  "city_name": "Mataram",
  "postal_code": "83111"
}
```

Field `province` dan `type` boleh kosong karena frontend hanya membutuhkan `city_id`, `city_name`, dan `postal_code`.

2.7 Menghitung Ongkos Kirim

```
public function getCost(Request $request): JsonResponse
{
    $validated = $request->validate([
        'origin' => ['required', 'integer', 'min:1'],
        'destination' => ['required', 'integer', 'min:1'],
        'weight' => ['required', 'integer', 'min:1', 'max:1000000'],
        'courier' => ['required', 'string', 'max:50', 'regex:/^[a-z]+(?::[a-z]+)*$/i'],
    ], [
        'origin.required' => 'Kota asal wajib dipilih.',
        'origin.integer' => 'Kota asal tidak valid.',
        'destination.required' => 'Kota tujuan wajib dipilih.',
        'destination.integer' => 'Kota tujuan tidak valid.',
        'weight.required' => 'Berat barang wajib diisi.',
        'weight.integer' => 'Berat barang harus berupa angka.',
        'weight.min' => 'Berat barang minimal 1 gram.',
        'weight.max' => 'Berat barang maksimal 1.000.000 gram.',
        'courier.required' => 'Kurir wajib dipilih.',
        'courier.regex' => 'Kode kurir tidak valid.',
    ]);

    return $this->handle(function () use ($validated): JsonResponse {
        $costs = $this->fetchCosts(
            (string) $validated['origin'],
            (string) $validated['destination'],
            (int) $validated['weight'],
            strtolower((string) $validated['courier'])
        );

        return $this->success($costs);
    });
}
```

Validasi ini penting karena:

- Berat 0 atau negatif akan ditolak API.
- `courier` dibatasi hanya huruf dan titik dua, sehingga tidak bisa dipakai untuk menyisipkan data aneh.
- `weight` dibatasi maksimal 1.000.000 gram agar request tidak berlebihan.

Bagian `fetchCosts()` untuk Komerce:

```
$payload = $this->request(
    fn (): Response => $this->client()->asForm()->post(
        $this->baseUrl().'/calculate/domestic-cost',
        [
            'origin' => $origin,
            'destination' => $destination,
            'weight' => $weight,
            'courier' => $courier,
            'price' => 'lowest',
        ]
    )
);

return collect($this->dataItems($payload))
    ->map(function (array $item) use ($courier): array {
        $cost = $item['cost'] ?? 0;

        if (is_array($cost)) {
```

```
            $cost = $cost[0]['value'] ?? 0;
        }

        return [
            'courier' => (string) ($item['code'] ?? $courier),
            'name' => (string) ($item['name'] ?? strtoupper($courier)),
            'service' => (string) ($item['service'] ?? ''),
            'description' => (string) ($item['description'] ?? ''),
            'cost' => (int) round((float) $cost),
            'etd' => $this->normalizeEtd($item['etd'] ?? ''),
            'note' => (string) ($item['note'] ?? ''),
        ];
    })
    ->filter(fn (array $item): bool => $item['service'] !== '')
    ->values()
    ->all();
```

Hasil normalisasi biaya:

```
{
  "courier": "jne",
  "name": "JNE",
  "service": "REG",
  "description": "Layanan Reguler",
  "cost": 14000,
  "etd": "2",
  "note": ""
}
```

2.8 Normalisasi Respons API

Normalisasi adalah proses mengubah bentuk data dari API menjadi bentuk yang seragam. Tanpa normalisasi, frontend harus menangani dua bentuk data yang berbeda.

Data	Komerce	RajaOngkir Starter	Hasil Normalisasi
ID provinsi	`data[].id`	`rajaongkir.results[].province_id`	`province_id`
Nama provinsi	`data[].name`	`rajaongkir.results[].province`	`province`
ID kota	`data[].id`	`rajaongkir.results[].city_id`	`city_id`
Nama kota	`data[].name`	`rajaongkir.results[].city_name`	`city_name`
Kode pos	`data[].zip_code`	`rajaongkir.results[].postal_code`	`postal_code`
Layanan	`data[].service`	`results[].costs[].service`	`service`
Biaya	`data[].cost`	`costs[].cost[].value`	`cost`
Estimasi	`data[].etd`	`costs[].cost[].etd`	`etd`

Fungsi `normalizeEtd()` menghapus akhiran `day` atau `days`:

```
private function normalizeEtd(mixed $etd): string
{
    $value = trim((string) $etd);

    return (string) preg_replace('/\s*days?$/i', '', $value);
}
```

Contoh: `"2 days"` menjadi `"2"`, sedangkan `"2-3"` tetap `"2-3"`.

2.9 Caching Data Ongkir

Data provinsi dan kota jarang berubah, sehingga cocok disimpan di cache selama satu hari.

```
$provinces = Cache::remember(
    $this->cacheKey('provinces'),
    now()->addDay(),
    fn (): array => $this->fetchProvinces()
);
```

Cache key dibuat dengan fingerprint API key:

```
private function cacheKey(string $suffix): string
{
    $keyFingerprint = substr(hash('sha256', $this->apiKey()), 0, 12);

    return 'rajaongkir.'.($this->isKomerce() ? 'komerce' : 'starter').'.'.$keyFingerprint.'.'.$suffix;
}
```

Manfaatnya:

- Mengurangi jumlah request ke API.
- Halaman lebih cepat.
- Jika API key diganti, cache lama tidak dipakai lagi.
- Data Komerce dan Starter tidak tercampur.

> **Catatan:** data biaya ongkir tidak di-cache karena biaya dapat berubah dan bergantung pada berat, tujuan, serta kurir.

2.10 Penanganan Error

Method `handle()` mengubah exception menjadi respons JSON yang jelas:

```
private function handle(callable $callback): JsonResponse
{
    if ($this->apiKey() === '') {
        return $this->error(
            'Kunci API RajaOngkir belum diatur. Isi RAJAONGKIR_API_KEY pada berkas .env.',
            503
        );
    }

    try {
        return $callback();
    } catch (ConnectionException $exception) {
        Log::warning('RajaOngkir: koneksi gagal.', [
            'message' => $exception->getMessage(),
        ]);

        return $this->error(
            'Tidak dapat menghubungi layanan RajaOngkir. Periksa koneksi internet lalu coba lagi.',
            503
        );
    } catch (RajaOngkirException $exception) {
```

```
            return $this->error($exception->getMessage(), $exception->httpStatus());
        } catch (Throwable $exception) {
            Log::error('RajaOngkir: kesalahan tak terduga.', [
                'message' => $exception->getMessage(),
            ]);

            return $this->error('Terjadi kesalahan saat memproses permintaan ongkir.', 500);
        }
    }
```

Tabel status error:

Kondisi	HTTP Status	Pesan
API key kosong	503	Kunci API belum diatur
Koneksi internet gagal	503	Tidak dapat menghubungi layanan
API key salah / tidak punya akses	502	API Key tidak valid atau tidak memiliki akses
API mengembalikan error lain	502	Pesan dari RajaOngkir
Kesalahan tak terduga	500	Terjadi kesalahan saat memproses permintaan

> **Keamanan:** pesan error tidak boleh menampilkan API key. Log hanya mencatat status dan pesan umum.

2.11 Route Ongkir

```
Route::get('/cek-ongkir', function () {
    return view('ongkir', [
        'judul' => 'Cek Ongkos Kirim',
    ]);
})->name('ongkir.index');

Route::get('/provinces', [RajaOngkirController::class, 'getProvinces'])
    ->name('ongkir.provinces');

Route::get('/cities', [RajaOngkirController::class, 'getCities'])
    ->name('ongkir.cities');

Route::post('/cost', [RajaOngkirController::class, 'getCost'])
    ->name('ongkir.cost');
```

Route ini bersifat publik karena halaman cek ongkir memang boleh diakses tanpa login. Namun, route checkout tetap berada di dalam middleware `is.customer`.

2.12 Halaman Cek Ongkir

Halaman `resources/views/ongkir.blade.php` berisi form sederhana:

```
<form id="ongkirForm">
    <select name="province" id="province">
        <option value="">Pilih Provinsi</option>
    </select>
    <select name="city" id="city">
        <option value="">Pilih Kota</option>
    </select>
    <input type="number" name="weight" id="weight" placeholder="Berat (gram)">
    <select name="courier" id="courier">
```

```
        <option value="">Pilih Kurir</option>
        <option value="jne">JNE</option>
        <option value="tiki">TIKI</option>
        <option value="pos">POS Indonesia</option>
    </select>
    <button type="submit">Cek Ongkir</button>
</form>
<div id="result"></div>
```

Alur JavaScript:

1. Saat halaman dimuat, ambil `/provinces`.
2. Saat provinsi dipilih, ambil `/cities?province_id=...`.
3. Saat form dikirim, kirim POST ke `/cost`.
4. Tampilkan layanan, biaya, dan estimasi.

Contoh pemanggilan biaya:

```
const response = await fetch('/cost', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json',
        'X-CSRF-TOKEN': csrfToken
    },
    body: JSON.stringify({
        origin: originCityId,
        destination: destinationCityId,
        weight: orderWeight,
        courier: courier
    })
});
```

2.13 Pengujian Minggu 7

File `tests/Feature/RajaOngkirTest.php` menguji:

1. Normalisasi provinsi Komerce.
2. Normalisasi kota Komerce.
3. Normalisasi biaya Komerce.
4. Dukungan RajaOngkir Starter.
5. API key kosong menghasilkan 503.
6. API key tidak valid menghasilkan pesan jelas.
7. Validasi input kota dan ongkir.

Hasil: **7 test lulus**.

Perintah menjalankan:

```
php artisan test --filter=RajaOngkirTest
```

Pengujian nyata dengan API Komerce juga berhasil:

```
Provinsi: 34 data
Kota: data kota berhasil diambil
```

```
Ongkir: JNE REG, biaya Rp14.000, estimasi 2 hari, berat 250 gram
```

2.14 Perbedaan Modul vs Implementasi

Aspek	Modul	Implementasi Proyek
Konfigurasi	`env()` langsung di controller	`config('services.rajaongkir.*')`
Format respons	Mengasumsikan `rajaongkir`	Mendukung Komerce dan Starter
Kota asal	Hardcode `501`	Config `RAJAONGKIR_ORIGIN_CITY_ID`
Validasi input	Belum ada	Validasi province, city, weight, courier
Caching	Belum ada	Cache provinsi/kota selama 1 hari
Error handling	Belum ada	503/502/500 dengan pesan jelas
Bentuk respons	Mengembalikan JSON mentah	`{ status, data }` yang konsisten
Keamanan key	Key di contoh kode	Key hanya di `.env`

2.15 Latihan

1. Ubah kota asal ke kota lain dan pastikan ongkir berubah.
2. Tambahkan kurir lain yang didukung API.
3. Ubah masa cache menjadi 6 jam.
4. Uji dengan API key yang salah dan amati pesan error.
5. Tambahkan tombol “Reset” untuk mengosongkan pilihan kota dan layanan.

Bagian 3 — Minggu ke-9: Proses Checkout Produk

3.1 Tujuan Pembelajaran

Setelah mempelajari bagian ini, kita mampu:

- Menghubungkan keranjang belanja dengan halaman pilih pengiriman.
- Menyimpan data ongkir yang dipilih customer.
- Menampilkan ringkasan pembayaran yang benar.
- Mengurangi stok produk secara aman saat checkout.
- Menyimpan dan menampilkan history pesanan.
- Menjaga agar customer hanya bisa melihat pesanannya sendiri.

3.2 Alur Checkout dan Status Pesanan

Alur status yang dipakai:

```
pending  ->  Paid  ->  Kirim  ->  Selesai
   |           |
   |           +-- masuk halaman history customer
   |
   +-- masih di keranjang, belum dibayar
```

Penjelasan:

- `pending` — pesanan masih berupa keranjang.
- `Paid` — customer sudah menekan tombol Bayar; pesanan masuk proses.
- `Kirim` — admin sudah mengirim pesanan dan mengisi nomor resi (Minggu 10).
- `Selesai` — pesanan selesai.

> **Penting:** modul menulis status `completed` saat checkout, tetapi halaman history memfilter `Paid`, `Kirim`, dan `Selesai`. Implementasi proyek memakai `Paid` agar pesanan langsung muncul di history dan konsisten dengan Minggu 10.

3.3 Database dan Model

Tabel `order` menyimpan data pesanan:

Kolom	Tipe	Keterangan
`id`	bigint	Primary key
`customer_id`	bigint	Relasi ke tabel customer
`user_id`	bigint nullable	Relasi ke tabel users
`status`	string	`pending`, `Paid`, `Kirim`, `Selesai`
`noresi`	string nullable	Nomor resi
`kurir`	string nullable	Kode/nama kurir
`layanan_ongkir`	string nullable	REG, YES, dll.
`biaya_ongkir`	string nullable	Biaya dari API
`estimasi_ongkir`	string nullable	Estimasi hari
`total_berat`	integer nullable	Total berat dalam gram
`total_harga`	double	Subtotal produk
`alamat`	text nullable	Alamat pengiriman
`pos`	string nullable	Kode pos
`timestamps`	timestamp	created_at, updated_at

Tabel `order_item` menyimpan detail produk:

Kolom	Tipe	Keterangan
`id`	bigint	Primary key
`order_id`	bigint	Relasi ke order
`produk_id`	bigint	Relasi ke produk
`quantity`	integer	Jumlah produk
`harga`	double	Harga saat dimasukkan keranjang
`timestamps`	timestamp	created_at, updated_at

Model `Order` menambahkan cast agar tipe data konsisten:

```
protected $casts = [
    'biaya_ongkir' => 'integer',
    'total_berat' => 'integer',
    'total_harga' => 'float',
];
```

Dengan cast ini, `$order->biaya_ongkir` selalu berupa integer, sehingga pengecekan `=== null` dan penjumlahan lebih aman.

3.4 Memilih Pengiriman

```
public function selectShipping(Request $request)
{
    $customer = $this->currentCustomer();
    $order = $customer ? $this->pendingOrder($customer) : null;
```

```
    if (! $order || $order->orderItems()->count() === 0) {
        return redirect()->route('order.cart')->with('error', 'Keranjang belanja kosong.');
    }

    $order->load('orderItems.produk');
    [$totalHarga, $totalBerat, $totalQuantity] = $this->orderTotals($order);

    return view('v_order.select_shipping', [
        'judul' => 'Pilih Pengiriman',
        'order' => $order,
        'totalHarga' => $totalHarga,
        'totalBerat' => $totalBerat,
        'totalQuantity' => $totalQuantity,
        'originCityId' => (string) config('services.rajaongkir.origin_city_id', '137'),
        'originCityName' => (string) config('services.rajaongkir.origin_city_name', 'Jakarta Pusat'),
    ]);
}
```

Yang perlu diperhatikan:

- Hanya pesanan `pending` milik customer yang boleh dibuka.
- Jika keranjang kosong, customer diarahkan kembali ke keranjang.
- Total berat dihitung dari produk dan quantity, bukan dari input user.
- Kota asal diambil dari konfigurasi, sehingga tidak bisa diubah dari browser.

3.5 Menyimpan Ongkir dengan Revalidasi Server

Ini bagian terpenting pada checkout. Jangan pernah percaya biaya ongkir yang dikirim dari form, karena user bisa memanipulasinya.

Validasi input:

```
$validated = $request->validate([
    'destination_city_id' => ['required', 'integer', 'min:1'],
    'city_name' => ['required', 'string', 'max:100'],
    'province_name' => ['required', 'string', 'max:100'],
    'kurir' => ['required', 'string', 'max:20', 'regex:/^[a-z]+$/i'],
    'layanan_ongkir' => ['required', 'string', 'max:50'],
    'alamat' => ['required', 'string', 'max:500'],
    'pos' => ['required', 'string', 'max:10'],
], [
    'destination_city_id.required' => 'Kota tujuan wajib dipilih.',
    'city_name.required' => 'Nama kota wajib diisi.',
    'province_name.required' => 'Nama provinsi wajib diisi.',
    'kurir.required' => 'Kurir wajib dipilih.',
    'layanan_ongkir.required' => 'Layanan pengiriman wajib dipilih.',
    'alamat.required' => 'Alamat lengkap wajib diisi.',
    'pos.required' => 'Kode pos wajib diisi.',
]);
```

Revalidasi ke API:

```
$costRequest = Request::create('/cost', 'POST', [
    'origin' => (string) config('services.rajaongkir.origin_city_id', '137'),
    'destination' => (string) $validated['destination_city_id'],
    'weight' => (int) $order->total_berat,
    'courier' => strtolower($validated['kurir']),
```

```
]);
$costRequest->headers->set('Accept', 'application/json');

$costResponse = app(RajaOngkirController::class)->getCost($costRequest);
$costPayload = json_decode($costResponse->getContent(), true);

if ($costResponse->getStatusCode() !== 200 || ($costPayload['status']['code'] ?? null) !== 200) {
    return back()->withInput()->with(
        'error',
        $costPayload['message'] ?? 'Gagal menghitung ongkos kirim. Silakan coba lagi.'
    );
}
```

Mencari layanan yang dipilih:

```
$selected = collect($costPayload['data'] ?? [])->first(function (array $item) use ($validated): bool {
    return strcasecmp((string) ($item['service'] ?? ''), (string) $validated['layanan_ongkir']) === 0
        && strcasecmp((string) ($item['courier'] ?? ''), (string) $validated['kurir']) === 0;
});

if (! $selected) {
    return back()->withInput()->with(
        'error',
        'Layanan pengiriman yang dipilih tidak tersedia. Silakan cek ongkir ulang.'
    );
}
```

Menyimpan data dari API, bukan dari form:

```
$order->update([
    'kurir' => strtoupper((string) $selected['courier']),
    'layanan_ongkir' => (string) $selected['service'],
    'biaya_ongkir' => (int) $selected['cost'],
    'estimasi_ongkir' => (string) $selected['etd'],
    'total_berat' => (int) $order->total_berat,
    'alamat' => trim($validated['alamat']).', '.$validated['city_name'].', '.$validated['province_name'],
    'pos' => $validated['pos'],
]);
```

> **Prinsip keamanan:** browser hanya mengirim *pilihan* (kota, kurir, layanan). Server tetap meminta ulang biaya ke API dan memakai biaya dari respons API. Dengan begitu, harga tidak bisa diubah lewat inspect element.

3.6 Halaman Pembayaran

```
public function selectPayment()
{
    $customer = $this->currentCustomer();
    $order = $customer ? $this->pendingOrder($customer) : null;

    if (! $order || $order->orderItems()->count() === 0) {
        return redirect()->route('order.cart')->with('error', 'Keranjang belanja kosong.');
    }

    if (! $order->kurir || $order->biaya_ongkir === null) {
        return redirect()->route('order.cart')
            ->with('error', 'Pilih pengiriman terlebih dahulu sebelum melakukan pembayaran.');
    }
```

```
    $order->load('orderItems.produk');
    [$subtotal] = $this->orderTotals($order);
    $biayaOngkir = (int) $order->biaya_ongkir;

    return view('v_order.selectpayment', [
        'judul' => 'Pembayaran',
        'order' => $order,
        'subtotal' => $subtotal,
        'biayaOngkir' => $biayaOngkir,
        'totalBayar' => $subtotal + $biayaOngkir,
    ]);
}
```

Halaman pembayaran menampilkan:

- Daftar produk dan quantity.
- Alamat pengiriman.
- Kurir dan layanan.
- Subtotal produk.
- Ongkos kirim.
- Total bayar.

Total bayar dihitung di server, bukan di JavaScript, sehingga nilai yang ditampilkan konsisten dengan data pesanan.

3.7 Checkout dan Pengurangan Stok

Checkout harus atomik: jika satu produk gagal, seluruh proses dibatalkan. Karena itu digunakan `DB::transaction()` dan `lockForUpdate()`.

```
DB::transaction(function () use ($order): void {
    $lockedOrder = Order::whereKey($order->id)->lockForUpdate()->first();

    if (! $lockedOrder || $lockedOrder->status !== 'pending') {
        throw new RuntimeException('Pesanan tidak ditemukan atau sudah diproses.');
    }

    $items = $lockedOrder->orderItems()->with('produk')->get();

    if ($items->isEmpty()) {
        throw new RuntimeException('Keranjang belanja kosong.');
    }

    foreach ($items as $item) {
        $produk = Produk::whereKey($item->produk_id)->lockForUpdate()->first();

        if (! $produk || $produk->status !== 'aktif') {
            throw new RuntimeException('Produk '.($item->produk->nama_produk ?? '').' tidak tersedia.');
        }

        if ((int) $produk->stok < (int) $item->quantity) {
            throw new RuntimeException('Stok produk '.$produk->nama_produk.' tidak mencukupi.');
        }

        $produk->update([
            'stok' => (int) $produk->stok - (int) $item->quantity,
        ]);
    }
```

```
    $this->recalculateOrder($lockedOrder);
    $lockedOrder->update(['status' => 'Paid']);
});
```

Mengapa perlu `lockForUpdate()`?

- Mencegah dua customer membeli stok terakhir secara bersamaan.
- Memastikan stok yang dibaca adalah stok terbaru.
- Menghindari stok menjadi minus.

Jika terjadi `RuntimeException`, transaksi di-rollback dan customer kembali ke keranjang dengan pesan error.

3.8 History Pesanan

```
public function orderHistory()
{
    $customer = $this->currentCustomer();

    if (! $customer) {
        return redirect()->route('beranda')->with('error', 'Data customer tidak ditemukan.');
    }

    $orders = Order::with(['orderItems.produk'])
        ->where('customer_id', $customer->id)
        ->where('user_id', Auth::id())
        ->whereIn('status', ['Paid', 'Kirim', 'Selesai'])
        ->orderByDesc('id')
        ->get();

    return view('v_order.history', [
        'judul' => 'History Pesanan',
        'orders' => $orders,
    ]);
}
```

Filter penting:

- `customer_id` — hanya pesanan milik customer.
- `user_id` — lapisan pengaman tambahan agar data tidak bocor.
- `whereIn('status', ...)` — keranjang `pending` tidak ditampilkan sebagai history.

3.9 Route Checkout

```
Route::middleware('is.customer')->group(function () {
    Route::match(['get', 'post'], '/select-shipping', [OrderController::class, 'selectShipping'])
        ->name('order.selectShipping');

    Route::post('/updateongkir', [OrderController::class, 'updateOngkir'])
        ->name('order.updateongkir');

    Route::get('/selectpayment', [OrderController::class, 'selectPayment'])
        ->name('order.selectPayment');

    Route::post('/checkout', [OrderController::class, 'checkout'])
        ->name('order.checkout');
```

```
    Route::get('/history', [OrderController::class, 'orderHistory'])
        ->name('order.history');
});
```

Semua route checkout berada di middleware `is.customer`, sehingga guest diarahkan ke login.

3.10 Tampilan Checkout

3.10.1 Halaman Pilih Pengiriman

File: `resources/views/v_order/select_shipping.blade.php`

Komponen utama:

- Form alamat: provinsi, kota, alamat lengkap, kode pos.
- Input berat total (disabled, dihitung server).
- Pilihan kurir: JNE, TIKI, POS Indonesia.
- Tombol **Cek Ongkir**.
- Tombol **Lanjut ke Pembayaran**.
- Tombol **Kembali ke Keranjang**.
- Ringkasan belanja di kolom kanan.
- Tabel layanan pengiriman yang muncul setelah cek ongkir.

Alur JavaScript:

```
DOMContentLoaded
    |
    +--> loadProvinces()
    |
    +--> pilih provinsi -> loadCities()
    |
    +--> pilih kota -> isi hidden city_id, city_name, province_name
    |
    +--> klik Cek Ongkir -> POST /cost
    |
    +--> render tabel layanan (radio)
    |
    +--> pilih layanan -> isi hidden layanan_ongkir
    |
    +--> submit -> validasi serviceCode -> POST /updateongkir
```

Hidden field yang dikirim ke server:

```
<input type="hidden" name="destination_city_id" id="destinationCityId">
<input type="hidden" name="city_name" id="cityName">
<input type="hidden" name="province_name" id="provinceName">
<input type="hidden" name="layanan_ongkir" id="serviceCode">
```

3.10.2 Halaman Pembayaran

File: `resources/views/v_order/selectpayment.blade.php`

Halaman ini menampilkan:

- Tabel produk pesanan.
- Alamat pengiriman.

- Ringkasan subtotal, ongkir, total bayar.
- Tombol **Bayar**.
- Tombol kembali ke pengiriman dan keranjang.

Tombol Bayar mengirim POST ke `order.checkout` dengan konfirmasi JavaScript:

```
<form action="{{ route('order.checkout') }}" method="post"
    onsubmit="return confirm('Konfirmasi pembayaran pesanan ini?');">
    @csrf
    <button type="submit" class="primary-btn" style="width: 100%;">Bayar</button>
</form>
```

3.10.3 Halaman History

File: `resources/views/v_order/history.blade.php`

Setiap pesanan ditampilkan dalam panel:

- ID pesanan dan tanggal.
- Label status: Diproses, Dikirim, Selesai.
- Tabel produk.
- Subtotal, ongkir, total bayar.
- Alamat pengiriman.
- Nomor resi (jika sudah ada).

Label status menggunakan `match`:

```
$statusLabel = match ($status) {
    'paid' => 'Diproses',
    'kirim' => 'Dikirim',
    'selesai' => 'Selesai',
    default => ucfirst($order->status),
};
```

3.11 Perbaikan Layout Tombol

Awalnya tombol **Cek Ongkir**, **Lanjut ke Pembayaran**, dan **Kembali ke Keranjang** ditumpuk dengan `margin-left`, sehingga tampak berantakan dan warnanya tidak seragam.

Perbaikan yang dilakukan:

```
.shipping-actions {
    display: flex;
    flex-wrap: wrap;
    margin: 20px -5px 0;
}

.shipping-actions .shipping-action-btn {
    flex: 1 1 180px;
    margin: 5px;
    text-align: center;
}

.shipping-back-btn {
    display: block;
    width: 100%;
```

```
    margin-top: 10px;
    text-align: center;
}
```

Hasilnya:

- **Cek Ongkir** dan **Lanjut ke Pembayaran** tampil berdampingan dengan lebar seimbang.
- **Kembali ke Keranjang** berada di baris bawah, full width.
- Ketiga tombol memakai warna utama yang sama.
- Di layar kecil, tombol otomatis turun baris.

3.12 Catatan Payment Gateway

Modul Minggu 9 menyinggung Midtrans, tetapi integrasi payment gateway termasuk materi lanjutan (Pertemuan 12). Pada implementasi ini:

- Tombol **Bayar** hanya mengonfirmasi pesanan.
- Status berubah dari `pending` menjadi `Paid`.
- Stok berkurang.
- Pesanan masuk history.

Halaman pembayaran menampilkan informasi bahwa payment gateway akan diintegrasikan pada pertemuan berikutnya. Dengan begitu, alur checkout tetap bisa diuji tanpa ketergantungan layanan pembayaran.

3.13 Pengujian Minggu 9

File `tests/Feature/OrderCheckoutTest.php` menguji:

1. Halaman select shipping menampilkan ringkasan keranjang.
2. Update ongkir menyimpan biaya dari API.
3. Update ongkir menolak layanan yang tidak tersedia.
4. Halaman pembayaran menampilkan subtotal, ongkir, dan total.
5. Checkout mengurangi stok dan menyimpan history.
6. Checkout ditolak jika stok tidak mencukupi.
7. History hanya menampilkan pesanan milik customer.
8. Guest diarahkan ke login untuk checkout.

Hasil: **8 test lulus, 48 assertion**.

Total seluruh proyek: **32 test lulus, 187 assertion**.

Perintah:

```
php artisan test
php artisan test --filter=OrderCheckoutTest
```

Pengujian alur nyata dengan API Komerce:

```
Kurir       : JNE
Layanan     : REG
Biaya       : Rp14.000
Estimasi    : 2 hari
Berat       : 250 gram
```

```
Status      : Paid
Stok produk : 5 -> 4
History     : menampilkan produk dan status "Diproses"
```

3.14 Perbedaan Modul vs Implementasi

Aspek	Modul	Implementasi Proyek
Biaya ongkir	Diambil dari input form	Divalidasi ulang ke API
Status checkout	`completed`	`Paid` agar konsisten dengan history
Stok	Dikurangi tanpa transaksi	Transaksi + `lockForUpdate()`
Validasi stok	Sebagian	Dicek ulang saat checkout
Otorisasi	Belum ketat	Filter `customer_id` + `user_id`
Route select shipping	POST	GET/POST agar bisa dibuka langsung
Method ongkir	Di OrderController	Dipisah ke RajaOngkirController
Halaman history	Tidak disertakan lengkap	Dibuat lengkap dengan status dan total
Payment gateway	Midtrans	Ditunda, tombol konfirmasi

3.15 Latihan

1. Tambahkan kurir lain pada halaman pilih pengiriman.
2. Tambahkan filter layanan termurah/termahal.
3. Buat halaman detail pesanan untuk customer.
4. Tambahkan tombol “Batalkan Pesanan” untuk status `pending`.
5. Integrasikan Midtrans pada halaman pembayaran.
6. Tambahkan test untuk memastikan customer tidak bisa membuka pesanan customer lain.

Bagian 4 — Ringkasan dan Operasional

4.1 Ringkasan Alur

```
[Produk] -> [Keranjang] -> [Pilih Pengiriman] -> [Cek Ongkir]
     -> [Pilih Layanan] -> [Pembayaran] -> [Bayar]
     -> [Stok berkurang] -> [Status Paid] -> [History]
```

Poin penting:

- Data provinsi/kota di-cache, data biaya tidak.
- Biaya ongkir selalu dihitung ulang di server.
- Stok dikurangi dalam transaksi database.
- History hanya menampilkan pesanan yang sudah dibayar.
- API key tidak pernah ditampilkan ke browser.

4.2 Troubleshooting

Masalah	Penyebab	Solusi
503 "Kunci API belum diatur"	`RAJAONGKIR_API_KEY` kosong	Isi `.env`, lalu `php artisan optimize:clear`
502 "API Key tidak valid"	Key salah atau tidak punya akses	Periksa key di dashboard Komerce
`.env` error "unexpected whitespace"	Nilai berisi spasi tanpa kutip	Bungkus dengan tanda kutip
"Kota asal tidak valid"	ID kota asal salah	Gunakan ID yang valid dari API
"Layanan tidak tersedia"	Layanan berubah / tidak ada	Klik Cek Ongkir ulang
"Stok tidak mencukupi"	Stok berubah setelah masuk keranjang	Perbarui quantity di keranjang
History kosong	Status masih `pending`	Selesaikan checkout sampai `Paid`
Perubahan view tidak muncul	View cache lama	`php artisan view:clear`
Perubahan `.env` tidak terbaca	Config cache lama	`php artisan config:clear`

4.3 Checklist Belajar

- [ ] Saya bisa menjelaskan perbedaan Komerce dan RajaOngkir Starter.
- [ ] Saya bisa membaca response API dan memetakan field-nya.
- [ ] Saya bisa menambahkan validasi pada request API.
- [ ] Saya bisa menjelaskan fungsi cache dan kapan cache harus dibersihkan.
- [ ] Saya bisa menjelaskan mengapa biaya ongkir tidak boleh dipercaya dari form.
- [ ] Saya bisa menjelaskan fungsi `DB::transaction()` dan `lockForUpdate()`.
- [ ] Saya bisa menulis test fitur untuk alur checkout.
- [ ] Saya bisa menangani error API dengan pesan yang ramah pengguna.

4.4 Command Reference

```
# Menjalankan seluruh test
php artisan test

# Menjalankan test tertentu
php artisan test --filter=OrderCheckoutTest
php artisan test --filter=RajaOngkirTest

# Membersihkan cache
php artisan optimize:clear
php artisan config:clear
php artisan view:clear

# Melihat route
php artisan route:list

# Format kode
php vendor/bin/pint
```

4.5 Daftar File

Dibuat:

- `app/Exceptions/RajaOngkirException.php`
- `resources/views/ongkir.blade.php`
- `resources/views/v_order/select_shipping.blade.php`
- `resources/views/v_order/selectpayment.blade.php`
- `resources/views/v_order/history.blade.php`
- `tests/Feature/RajaOngkirTest.php`
- `tests/Feature/OrderCheckoutTest.php`

Diubah:

- `app/Http/Controllers/RajaOngkirController.php`
- `app/Http/Controllers/OrderController.php`
- `app/Models/Order.php`
- `config/services.php`
- `routes/web.php`
- `resources/views/v_order/cart.blade.php`
- `resources/views/v_layouts/app.blade.php`
- `.env`
- `.env.example`

4.6 Catatan Keamanan

1. **Jangan commit `.env`.** Pastikan `.env` ada di `.gitignore`.
2. **Jangan tampilkan API key** di halaman, log, atau pesan error.
3. **Validasi semua input** yang menuju API.
4. **Hitung ulang biaya ongkir di server** sebelum menyimpan pesanan.

5. **Gunakan transaksi dan lock** saat mengubah stok.
6. **Filter data berdasarkan user/customer** agar tidak bocor.
7. **Batasi akses route** dengan middleware yang sesuai.

Lampiran

A. Contoh Request dan Response Komerce

Request provinsi:

```
GET https://rajaongkir.komerce.id/api/v1/destination/province
key: your_api_key
Accept: application/json
```

Response yang dinormalisasi:

```
{
  "status": { "code": 200, "description": "OK" },
  "data": [
    { "province_id": "1", "province": "Bali" },
    { "province_id": "2", "province": "Bangka Belitung" }
  ]
}
```

Request biaya:

```
POST https://rajaongkir.komerce.id/api/v1/calculate/domestic-cost
key: your_api_key
Content-Type: application/x-www-form-urlencoded

origin=137&destination=1&weight=250&courier=jne&price=lowest
```

Response yang dinormalisasi:

```
{
  "status": { "code": 200, "description": "OK" },
  "data": [
    {
      "courier": "jne",
      "name": "JNE",
      "service": "REG",
      "description": "Layanan Reguler",
      "cost": 14000,
      "etd": "2",
      "note": ""
    }
  ]
}
```

B. Contoh Response RajaOngkir Starter

```
{
  "rajaongkir": {
    "status": { "code": 200, "description": "OK" },
    "results": [
      {
        "code": "jne",
        "name": "JNE",
        "costs": [
          {
```

```
            "service": "REG",
            "description": "Layanan Reguler",
            "cost": [
              { "value": 14000, "etd": "2-3", "note": "" }
            ]
          }
        ]
      }
    ]
  }
}
```

Controller mengubah bentuk ini menjadi bentuk normalisasi yang sama dengan Komerce.

C. Struktur Tabel Order

```
CREATE TABLE `order` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `customer_id` bigint unsigned NOT NULL,
  `user_id` bigint unsigned DEFAULT NULL,
  `status` varchar(255) NOT NULL,
  `noresi` varchar(255) DEFAULT NULL,
  `kurir` varchar(255) DEFAULT NULL,
  `layanan_ongkir` varchar(255) DEFAULT NULL,
  `biaya_ongkir` varchar(255) DEFAULT NULL,
  `estimasi_ongkir` varchar(255) DEFAULT NULL,
  `total_berat` int DEFAULT NULL,
  `total_harga` double NOT NULL,
  `alamat` text,
  `pos` varchar(255) DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `order_customer_id_foreign` (`customer_id`),
  KEY `order_user_id_foreign` (`user_id`),
  CONSTRAINT `order_customer_id_foreign` FOREIGN KEY (`customer_id`) REFERENCES `customer` (`id`) ON
DELETE CASCADE,
  CONSTRAINT `order_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

D. Ringkasan Hasil Pengujian

Kelompok	Jumlah	Assertion	Status
RajaOngkir	7	-	Lulus
Checkout	8	48	Lulus
Seluruh proyek	32	187	Lulus

Pengujian nyata Komerce:

Item	Hasil
Kota asal	Jakarta Pusat (137)
Kota tujuan	Mataram (1)
Berat	250 gram
Kurir	JNE
Layanan	REG
Biaya	Rp14.000
Estimasi	2 hari
Status setelah bayar	Paid
Stok	5 menjadi 4
History	Menampilkan produk dan status Diproses

> **Selamat belajar.** Fokus pada alur data, bukan hanya sintaks. Jika alur data dipahami, kode akan lebih mudah diubah dan dikembangkan.